

Quality of Service mechanisms and the road to Quality of Experience

440-2216/01 Quality of Service • Lecture 03

Jan Rozhon, Miroslav Vozňák • Department of Telecommunications, FEECS

Where we are going

Part 1

QoE and QoS

Definitions, scope, and the four QoS parameters

↔ Step 1, Q1

Part 2

Delay budget

Fixed vs variable delay components

↔ Step 3, Q3

Part 3

Congestion

CAC, token bucket, WFQ, RED

↔ Steps 3–4, Q4

Part 4

Link efficiency

Compression, LFI, DSCP marking

↔ Question 2

One question runs through the lecture: **which mechanism changes what a network provides, and does that change what a person perceives?**

What you should be able to do

1. State the ITU-T definitions of QoS and QoE and explain how their scope and focus differ.
2. Decompose an end-to-end delay budget into fixed and variable components and identify which one dominates under load.
3. Explain what CAC, token-bucket enforcement, WFQ scheduling, and RED each do, and why RED does not help non-responsive UDP/RTP traffic.
4. Explain why link fragmentation/interleaving and header compression matter on slow links, and how DSCP marking classifies traffic for a scheduler.

Keep asking: **which QoS parameter does this mechanism change — throughput, delay, jitter, or loss — and for whom?**

A network can only ever manage Quality of Service

Quality of Experience is what a person makes of it — QoS mechanisms are the tools this lecture builds

Part 1

Quality of Service and Quality of Experience

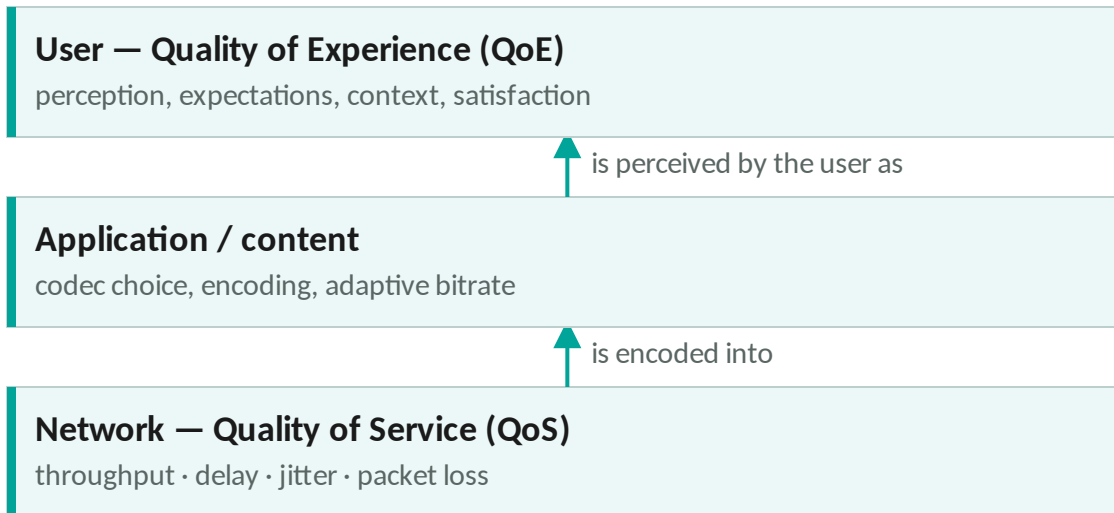
Quality of Service — a formal definition

ITU-T Recommendation E.800 defines Quality of Service as the

"Totality of characteristics of a telecommunications service that bear on its ability to satisfy stated and implied needs of the user of the service."

Traditionally, technology-centric approaches based on QoS parameters — throughput, delay, jitter, packet loss — have been used to ensure service quality. They are **objective, measurable at the network layer**, and the subject of the rest of this lecture.

Quality of Experience — a formal definition



QoS parameters are necessary inputs to QoE, but they are not sufficient on their own — expectations and context shape how the same measured QoS is experienced.

QoE is "the degree of delight or annoyance of the user of an application or service" (Qualinet White Paper, 2013; ITU-T P.10/G.100 Amd. 5). It expands the horizon beyond network performance to people's expectations and even hedonic needs.

Scope and focus: how QoS and QoE differ

Scope — QoS typically focuses on telecommunications services. QoE covers a much broader domain that sometimes does not involve telecommunications at all, e.g. HD video played in a home theatre.

Focus — QoS deals with performance aspects of physical systems. QoE deals with the user's *assessment* of that performance, as colored by context, culture, expectations and their fulfillment, socio-economic factors, and psychological profile, among other factors.

Both readings matter for Lab 03: the network side is what `tc netem` controls; the human side is what the ACR/DCR listening tests capture.

The QoS parameters this course measures

QoS features affect a network by manipulating four characteristics:

- **Throughput** — data delivered per unit time
- **Delay** — one-way latency from sender to receiver
- **Jitter** — delay *variation* between consecutive packets
- **Packet loss** — packets discarded or corrupted in transit

These four are exactly what Lab 03's `tc netem` injects (`delay` , a jitter argument, and `loss`) and exactly what PESQ and ViSQOL then score the perceptual effect of.

Checkpoint — is this a QoS problem?

A user complains a video call "feels bad", but monitoring shows 0% packet loss, jitter comfortably inside the receiver's buffer, and ample throughput. Is this necessarily a QoS problem?

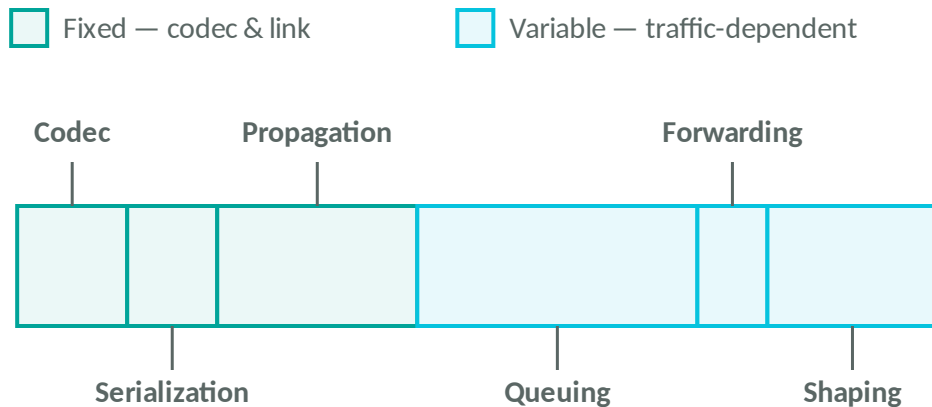
Predict first · discuss with a neighbour · then reveal

Not necessarily. All four measured QoS parameters look fine, yet QoE can still be poor for reasons outside them entirely: codec artifacts, screen or speaker hardware, poor lighting for video, mismatched expectations, or an urgent call versus a casual one. This is the scope/focus slide made concrete — good QoS numbers are necessary but not sufficient for good QoE.

Part 2

The one-way delay budget

Fixed delay: set once the link and codec are chosen



Fixed components are set once the codec and link are chosen; variable components grow under load.
ITU-T G.114 recommends keeping total one-way delay below about 150 ms for good conversational quality.

Serialization — time to encode a packet's bits onto the physical interface: packet size over link rate.

Propagation — time for one bit to cross the link, close to the speed of light in the medium.

Codec / packetization — time to encode a frame and accumulate enough samples to fill one packet.

Variable delay: grows under load

Queuing delay — time spent waiting in a router's output buffer before transmission begins.

Forwarding/processing delay — time to decide where to send a packet, typically small enough to ignore in a delay budget.

Shaping delay — added deliberately to match a contracted rate, so a carrier does not discard traffic that exceeds it.

Network delay — delay contributed by the carrier's own internal network components.

All four depend on **how busy the path is right now** — this is where Lecture 02's queueing theory re-enters the picture.

Worked example — a VoIP call's delay budget

G.711 call from Lab 03: 160 B payload, 40 B of RTP/UDP/IP headers, 20 ms packetization, over a 64 kbit/s branch-office link, 1500 km path.

Component	Delay
Codec/packetization (fixed)	20 ms
Serialization: 1600 bit / 64 kbit/s (fixed)	25 ms
Propagation: ≈ 1500 km (fixed)	7.5 ms
Queuing, lightly loaded (variable)	5 ms

Total ≈ 57.5 ms

well inside ITU-T G.114's ≈ 150 ms one-way guidance

Now let the link get busy: queuing delay alone climbs to 100 ms (Lecture 02: $W = 1/(\mu - \lambda)$ as $\rho \rightarrow 1$). Total becomes **157.5 ms** — over budget, with every fixed component unchanged.

Checkpoint — the dangerous component

Recall Lecture 02: $W = 1/(\mu - \lambda)$ grows without bound as $\rho \rightarrow 1$. Which single component of this delay budget behaves that way, and what does it imply for link dimensioning?

Predict first · discuss with a neighbour · then reveal

Queuing delay — the only component governed by an M/M/1-style formula that diverges as utilization approaches 1. Serialization, propagation, and codec delay are fixed once the link, path, and codec are chosen; shaping and network delay move only mildly. This is exactly why Lecture 02 stressed running links well below full utilization: near saturation, queuing delay alone can consume a delay budget that every other component left comfortably under threshold.

Part 3

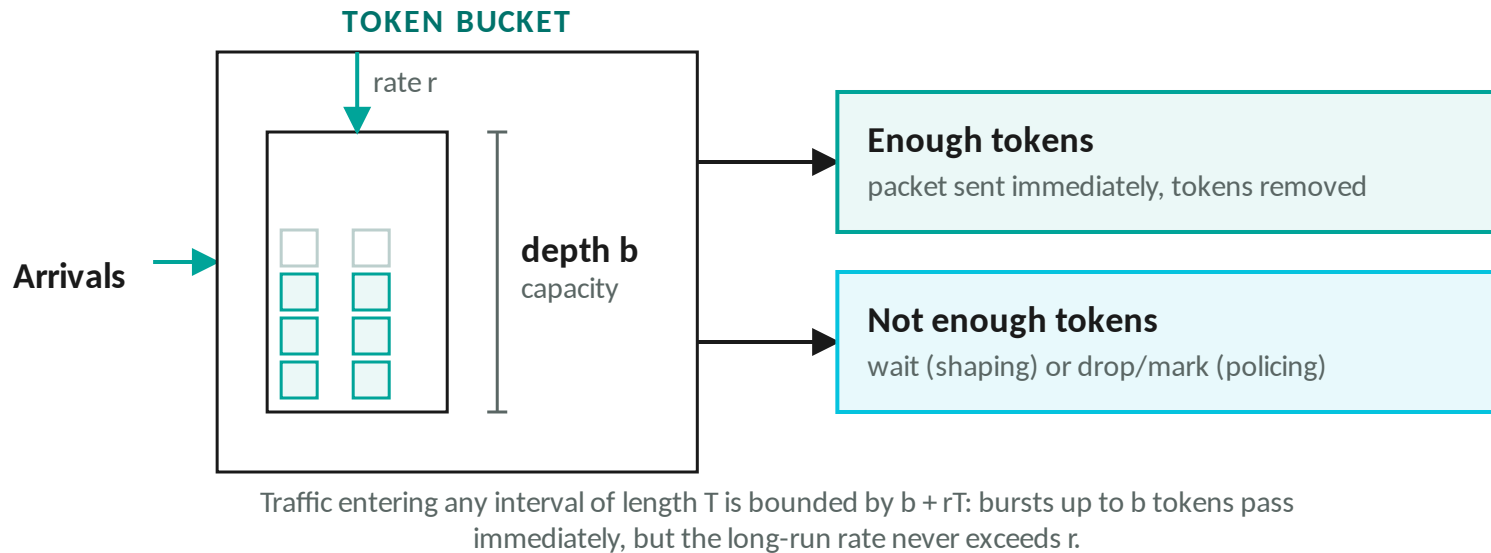
Managing congestion: admission, enforcement, scheduling, and avoidance

Call Admission Control (CAC)

CAC tools decide whether a network can accept a new voice or video call **before it starts**. That decision might rest on many factors, but several of them involve a measurement or estimate of available throughput.

This is Lecture 02's Erlang B logic, applied online: a loss system (M/M/N/N) decides per arriving call whether capacity remains, rather than dimensioning a trunk group offline from an average traffic figure.

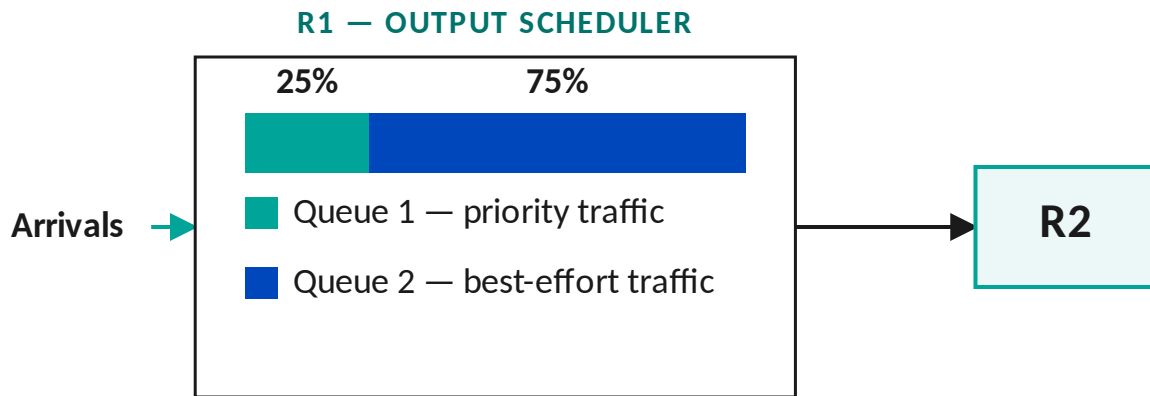
Traffic enforcement — the token bucket



Tokens fill a bucket at rate r , up to a maximum depth b . Sending a packet costs one token per byte. Traffic entering any interval of length T is bounded by $b + rT$: bursts up to b pass immediately, but the long-run rate never exceeds r .

Shaping delays a non-conformant packet until tokens accumulate (Part 2's "shaping delay"); **policing** instead drops or marks it immediately, without adding delay.

Queuing and scheduling — Weighted Fair Queuing



The scheduler serves queue 2 three times as often as queue 1 (75:25) — proportional sharing, not strict priority, so queue 1 is never starved of bandwidth.

Queuing tools create multiple queues; a **scheduling algorithm** decides which queue to serve next. **Weighted Fair Queuing (WFQ)** gives each queue a guaranteed minimum share — on a 1 Mbit/s link, the example above guarantees queue 1 at least 250 kbit/s and queue 2 at least 750 kbit/s.

This is **proportional** scheduling, not strict priority — queue 1 is never starved, only rate-limited relative to queue 2.

Why not just drop-tail? Three failure modes

Lock-out

A single flow's burst can fill the queue first and keep refilling it, leaving no room for any other flow's packets.

Full-queue bias

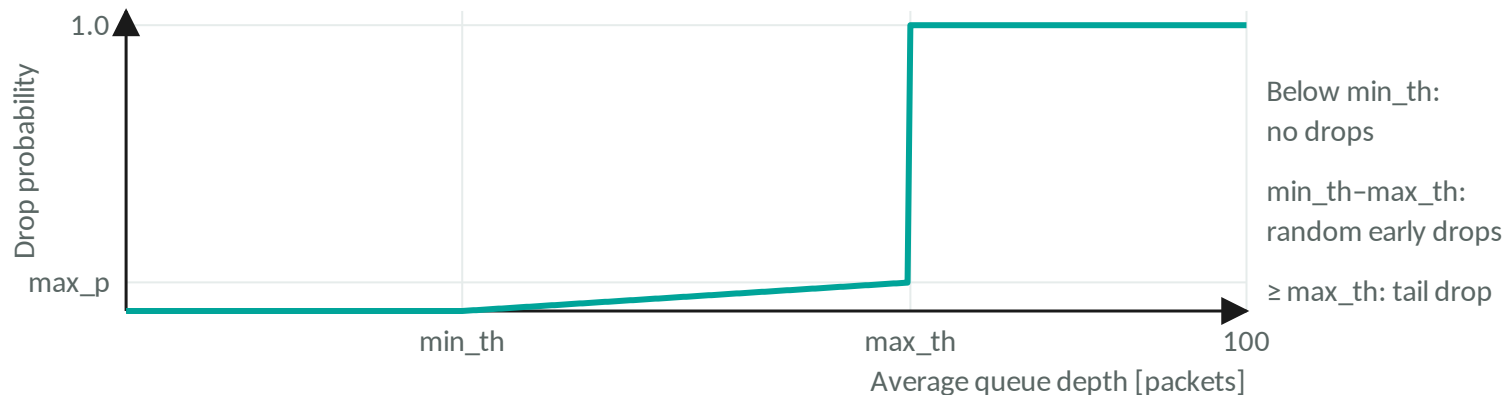
Packets are dropped only once the buffer is completely full — every flow suffers the maximum possible queuing delay before anything is dropped.

Global synchronization

Many TCP flows overflow the buffer at the same instant, all lose a packet together, and all back off and speed up in lockstep — causing periodic dips in link utilization.

An **active queue management** scheme — dropping before the buffer fills, not after — can avoid all three.

Congestion avoidance — Random Early Detection (RED)



Early, random drops signal congestion before the buffer fills. A TCP sender backs off in response; a UDP/RTP voice stream does not — it keeps sending at the same rate regardless.

RED discards some packets **before** a queue fills, with drop probability $p(\text{avg})$ rising linearly between the two thresholds shown above. avg is a **smoothed average** queue size — a short burst passes untouched, but a sustained trend still triggers drops.

An ECN-capable receiver (RFC 3168) can be **marked** instead of dropped — but this still relies on the sender responding to the signal.

RED and real-time media don't mix well

RED's whole mechanism assumes the sender **responds** to a dropped packet by slowing down — true for TCP, **not** true for RTP over UDP.

A TCP flow hit by RED

Loses a packet, halves its window, offers less load. RED's early warning does its job.

An RTP/UDP flow hit by RED

Loses a packet — degraded audio or video, packet-loss concealment at best — and keeps sending at exactly the same rate. Nothing about the congestion improves.

This is why real-time media additionally needs a **priority** discipline (a low-latency or strict-priority queue) rather than relying on RED alone — Part 4 revisits this once DSCP marking identifies which traffic that queue should protect.

Checkpoint — who actually backs off?

A congested router runs RED only, no priority queuing. A VoIP call and a large FTP transfer share the same queue. Whose packets get dropped, and does dropping either one relieve the congestion the way RED intends?

Predict first · discuss with a neighbour · then reveal

RED drops from both flows in proportion to their share of the queue — it does not distinguish them. The FTP's TCP stack backs off after a drop, relieving congestion exactly as RED intends. The VoIP call's RTP/UDP stream does not back off: its lost packets degrade the call without reducing its offered load at all. RED protects the network from responsive (TCP) flows; it does not protect a real-time flow from the network.

Fair Queuing vs. RED — two ways to protect a flow

(Weighted) Fair Queuing

Separates flows into their own queues, so one flow's misbehavior cannot consume another's guaranteed share. Real **isolation** — at the cost of per-flow or per-class state and classification.

RED on a shared queue

Needs no per-flow state — cheap to run at every hop. Provides **no isolation**: a non-adaptive flow is dropped at the same rate as everyone else and simply keeps going.

This is why real-time media is usually placed in its **own** priority queue (Part 4's DSCP marking identifies which packets belong there), rather than trusting RED alone to protect it on a shared queue.

Part 4

Link efficiency and marking

Compression and the cost of packet headers

Lab 03's G.711 packet: 160 B payload plus 40 B of RTP + UDP + IP headers = 200 B, so headers alone are **20% overhead** — significant on a slow access link.

RTP header compression (cRTP, RFC 2508) exploits how little those header fields change packet to packet on a point-to-point link, compressing 40 B down to as little as 2–4 B.

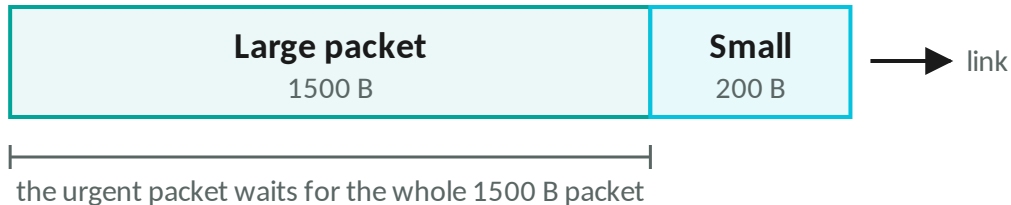
200 B → ≈162 B

overhead drops from 20% to about 1–2%

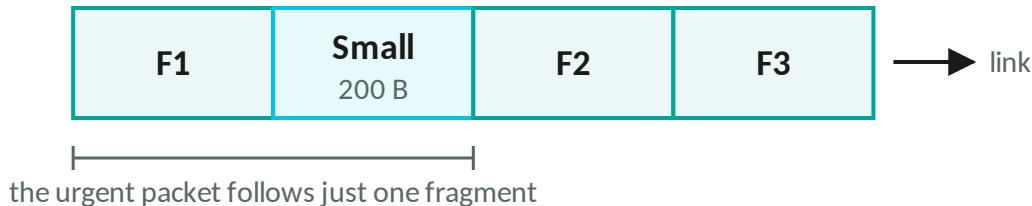
Compression is not free: it costs CPU time at both ends of the link (the "compression delay" from Part 2's fixed-delay list) — a trade worth making exactly on the slow links where serialization delay already matters.

Link Fragmentation and Interleaving (LFI)

WITHOUT LFI



WITH LFI (FRAGMENTATION + INTERLEAVING)



Once a router starts sending a packet, it finishes it — a delay-sensitive packet arriving just after a large one has begun transmission must wait for the whole thing. LFI fragments the large packet and interleaves the small, urgent one between fragments.

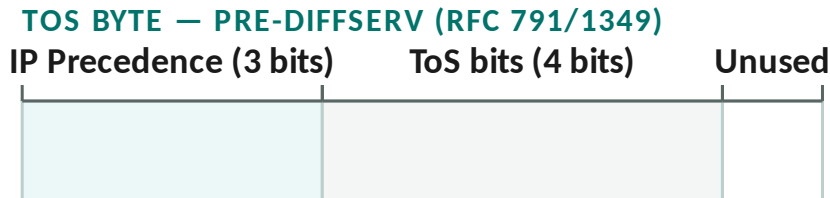
Standardized for multilink PPP (RFC 1990) and Frame Relay (FRF.12) — this bounds serialization delay's effect on someone else's queuing delay.

Classification and marking — Class of Service

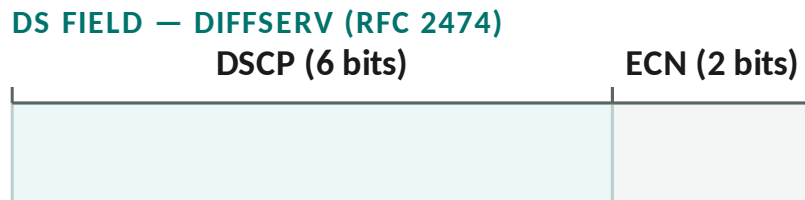
Classification and marking tools do the (potentially expensive) work of classifying a packet **once**, then mark a field in its header so every later device can act on that mark cheaply.

At Layer 2, **CoS** (Class of Service) is carried in the 802.1Q tag's 3-bit User Priority field, defined by IEEE 802.1p — meaningful only between devices that share a VLAN trunk.

Classification and marking — IP Precedence and DSCP



Same 8 bits, reinterpreted:



Class-selector codepoints (000xxx) preserve IP Precedence's ordering; DSCP adds finer-grained per-hop behaviors: EF = 101110 (voice, RFC 3246) · AF41 = 100010 (video, RFC 2597) · default = 000000.

RFC 2474/2475 redefine the same byte as **DSCP**, with standard per-hop behaviors (PHBs): **EF** for voice, **AF** for video, **CS0**/default for best-effort.

Marking only **classifies** — it changes nothing until a scheduler (Part 3's priority queue) is configured to treat EF-marked traffic differently.

Checkpoint — does marking alone help?

An engineer marks all VoIP traffic DSCP EF, but the congested interface still uses plain FIFO queuing. Does the marking help?

Predict first · discuss with a neighbour · then reveal

Not by itself. DSCP EF is a classification label; nothing enforces different treatment unless a scheduler is configured to read that label and act on it — for example, a low-latency queue that always serves EF-marked packets first. Marking and scheduling are two separate mechanisms that must be configured together, exactly as CAC and queuing are two separate decisions (admit vs. treat).

Common mix-ups

Marking vs. enforcing

DSCP/CoS labels a packet. Only a configured scheduler changes how it is actually treated.

RED vs. tail drop

RED drops early to warn TCP before the buffer is full. Tail drop happens once it is full — and can synchronize many flows' backoffs.

CAC vs. queuing

CAC decides whether a flow is admitted **at all**. Queuing/scheduling manage flows already admitted.

Blocked vs. delayed vs. dropped

CAC-rejected: never starts. Queued: delayed, not lost. RED/tail-dropped: discarded outright.

Shaping vs. policing

Both enforce a token-bucket contract. Shaping **delays** non-conformant traffic; policing **drops or marks** it.

Summary — seven mechanisms to keep straight

QoS $\Rightarrow$ QoE, not the reverse Throughput, delay, jitter, and loss are necessary but not sufficient inputs to QoE.

The delay budget Fixed (serialization + propagation + codec) plus variable (queuing + forwarding + shaping) — queuing is the one that blows up under load.

CAC vs. enforcement vs. scheduling CAC admits or blocks a flow; the token bucket holds it to a rate/burst contract; WFQ isolates queues from each other.

RED needs three failure modes to motivate it Drop-tail suffers lock-out, full-queue bias, and global synchronization — RED drops early and probabilistically to avoid all three.

RED does not help real-time media Non-responsive UDP/RTP traffic needs priority/WFQ queuing instead — RED buys cheap signaling, not isolation.

Shaping vs. policing Both enforce a token-bucket contract; shaping delays non-conformant traffic, policing drops or marks it.

Link efficiency LFI bounds serialization delay on slow links; header compression removes fixed per-packet overhead.

Marking needs a scheduler CoS/DSCP classify traffic; treatment changes only once a configured scheduler honors the marking.

Exit questions — use the ideas

1. A branch office's WAN link runs at 64 kbit/s. From this lecture's delay budget, which two components dominate, and how do they scale if the link is downgraded to 32 kbit/s?
2. Why is RED, by itself, an incomplete strategy for protecting a VoIP call sharing a link with bulk TCP downloads?
3. A packet arrives marked DSCP CS0 (best-effort). What, if anything, changes automatically about how a router queues it?
4. Contrast what happens to a call rejected by CAC, a packet delayed in a queue, and a packet dropped by RED.

Explain the mechanism, not just its name.

Lab 03 — where these mechanisms meet measured QoE

This lecture explained **where** delay, jitter, and loss come from on a real network. Lab 03 injects them **synthetically** and measures what a listener actually hears:

- Delay, jitter, loss ↔ `tc netem` parameters (Step 3)
- Header overhead and bit rate ↔ RTP/UDP/IP arithmetic (Question 2)
- The resulting audio ↔ PESQ and ViSQOL scoring (Steps 5–6)

Then the README questions connect network-layer causes (this lecture) to perceptual effects (the listening tests) — including whether *bursty* loss at the same average rate sounds worse than independent loss.

```
1  ssh <lab-server>
2  cd qos-03 && uv sync
3  uv run jupyter lab --ip 0.0.0.0
```

References — foundations

- ITU-T Recommendation E.800, *Definitions of terms related to quality of service*, 2008.
- ITU-T Recommendation P.10/G.100 Amendment 5, *New definition of Quality of Experience*, 2017.
- L. U. Rehman Khan, K. Connelly, N. Crespi, "Toward total quality of experience: A QoE model in a communication ecosystem," *IEEE Communications Magazine*, 50(4), art. 6178834, 58–65, 2012.
- Qualinet, *Qualinet White Paper on Definitions of Quality of Experience*, European Network on Quality of Experience in Multimedia Systems and Services (COST Action IC 1003), 2013.
- ITU-T Recommendation G.114, *One-way transmission time*, 2003.
- S. Floyd, V. Jacobson, "Random Early Detection Gateways for Congestion Avoidance," *IEEE/ACM Transactions on Networking*, 1(4), 397–413, 1993.

References — standards and practice

- K. Nichols, S. Blake, F. Baker, D. Black, "Definition of the Differentiated Services Field (DS Field) in the IPv4 and IPv6 Headers," RFC 2474, 1998.
- S. Blake et al., "An Architecture for Differentiated Services," RFC 2475, 1998.
- B. Davie et al., "An Expedited Forwarding PHB," RFC 3246, 2002.
- J. Heinanen, F. Baker, W. Weiss, J. Wroclawski, "Assured Forwarding PHB Group," RFC 2597, 1999.
- K. Sklower, B. Lloyd, G. McGregor, D. Carr, T. Coradetti, "The PPP Multilink Protocol (MP)," RFC 1990, 1996.
- S. Casner, V. Jacobson, "Compressing IP/UDP/RTP Headers for Low-Speed Serial Links," RFC 2508, 1999.
- T. Szigeti, C. Hattingh, *End-to-End QoS Network Design*. Cisco Press, 2004.

Thank you for your attention

Jan Rozhon, Miroslav Vozňák

+420 596 995 900 · jan.rozhon@vsb.cz

www.vsb.cz

Extras

Header compression details and a QoS standards timeline

Extras — how cRTP actually behaves

RFC 2508 compression runs **hop-by-hop** between one compressor/decompressor pair sharing per-flow context state — it only helps on that one link, not end-to-end.

Why it works

Most IP/UDP/RTP header fields are static or change predictably packet to packet (sequence number and timestamp both increment by a fixed step) — only the small delta needs to be sent.

Why it can fail

If the *link itself* loses a packet, the decompressor's context can desynchronize from the compressor's, corrupting every packet after it until a full header is resent.

Extras — a QoS/DiffServ standards timeline

Year	Standard	Contribution
1981	RFC 791	The IPv4 header's Type of Service byte
1994	RFC 1633	Integrated Services (IntServ) — per-flow reservations
1998	RFC 2474 / 2475	Differentiated Services (DiffServ) — per-hop behaviors, not per-flow state
1999	RFC 2597	Assured Forwarding (AF) PHB group
2002	RFC 3246	Expedited Forwarding (EF) PHB
2003	ITU-T G.114	One-way delay guidance for conversational quality

DiffServ won out over IntServ for wide deployment because it needs no per-flow state in core routers — a PHB is a property of a marking, not of a reservation that must be signaled and maintained hop by hop.